Validate the token

Learn how to securely validate Turnstile tokens on your server using the Siteverify API.

> **Mandatory server-side validation**
> You must call the Siteverify API to complete your Turnstile implementation. The client-side widget alone does not protect your forms.
>
> Server-side validation is required because:
>
> - **Tokens can be forged.** An attacker can submit any string to your form endpoint without completing a challenge.
> - **Tokens expire.** Each token is valid for 300 seconds (5 minutes) after generation.
> - **Tokens are single-use.** Each token can only be validated once. A replayed token will be rejected with the `timeout-or-duplicate` error code.

Process

1. Client generates token: Visitor completes Turnstile challenge on your webpage.
2. Token sent to server: Form submission includes the Turnstile token.
3. Server validates token: Your server calls Cloudflare's Siteverify API.
4. Cloudflare responds: Returns `success` or `failure` and additional data.
5. Server takes action: Allow or reject the original request based on validation.

Siteverify API overview

	Endpoint
	`POST https://challenges.cloudflare.com/turnstile/v0/siteverify`

Request format

The API accepts both `application/x-www-form-urlencoded` and `application/json` requests, but always returns JSON responses.

Required parameters

Parameter	Required	Description
`secret`	Yes	Your widget's secret key from the Cloudflare dashboard
`response`	Yes	The token from the client-side widget
`remoteip`	No	The visitor's IP address
`idempotency_key`	No	A UUID you generate to safely retry validation requests

Token characteristics

- Maximum length: 2048 characters
- Validity period: 300 seconds (5 minutes) from generation
- Single use: Each token can only be validated once
- Automatic expiry: Tokens automatically expire and cannot be reused

The validation token issued by Turnstile is valid for five minutes. If a user submits the form after this period, the token is considered expired. In this scenario, the server-side verification API will return a failure, and the `error-codes` field in the response will include `timeout-or-duplicate`.

To ensure a successful validation, the visitor must initiate the request and submit the token to your backend within the five-minute window. Otherwise, the Turnstile widget needs to be refreshed to generate a new token. This can be done using the `turnstile.reset` function.

Basic validation examples

JS JavaScript | PHP | Python | Java | C#

JSON

JavaScript

```javascript
const SECRET_KEY = "your-secret-key";

async function validateTurnstile(token, remoteip) {
  try {
    const response = await fetch(
      "https://challenges.cloudflare.com/turnstile/v0/siteverify",
      {
        method: "POST",
        headers: {
          "Content-Type": "application/json",
        },
        body: JSON.stringify({
          secret: SECRET_KEY,
          response: token,
          remoteip: remoteip,
        }),
      },
    );

    const result = await response.json();
    return result;
  } catch (error) {
    console.error("Turnstile validation error:", error);
    return { success: false, "error-codes": ["internal-error"] };
  }
}
```

Form Data

JavaScript

```
const SECRET_KEY = "your-secret-key";

async function validateTurnstile(token, remoteip) {
  const formData = new FormData();
  formData.append("secret", SECRET_KEY);
  formData.append("response", token);
  formData.append("remoteip", remoteip);

  try {
    const response = await fetch(
      "https://challenges.cloudflare.com/turnstile/v0/siteverify",
      {
        method: "POST",
        body: formData,
      },
    );

    const result = await response.json();
    return result;
  } catch (error) {
    console.error("Turnstile validation error:", error);
    return { success: false, "error-codes": ["internal-error"] };
  }
}

// Usage in form handler
async function handleFormSubmission(request) {
  const body = await request.formData();
  const token = body.get("cf-turnstile-response");
  const ip =
    request.headers.get("CF-Connecting-IP") ||
    request.headers.get("X-Forwarded-For") ||
    "unknown";

  const validation = await validateTurnstile(token, ip);

  if (validation.success) {
    // Token is valid - process the form
    console.log("Valid submission from:", validation.hostname);
    return processForm(body);
  } else {
    // Token is invalid - reject the submission
```

```
    console.log("Invalid token:", validation["error-codes"]);
    return new Response("Invalid verification", { status: 400 });
  }
}
```

Advanced validation techniques

Idempotency keys for retry operation

```
const crypto = require("crypto");

async function validateWithRetry(token, remoteip, maxRetries = 3) {
  const idempotencyKey = crypto.randomUUID();

  for (let attempt = 1; attempt <= maxRetries; attempt++) {
    try {
      const formData = new FormData();
      formData.append("secret", SECRET_KEY);
      formData.append("response", token);
      formData.append("remoteip", remoteip);
      formData.append("idempotency_key", idempotencyKey);

      const response = await fetch(
        "https://challenges.cloudflare.com/turnstile/v0/siteverify",
        {
          method: "POST",
          body: formData,
        },
      );

      const result = await response.json();

      if (response.ok) {
        return result;
      }

      // If this is the last attempt, return the error
      if (attempt === maxRetries) {
        return result;
      }

      // Wait before retrying (exponential backoff)
      await new Promise((resolve) =>
        setTimeout(resolve, Math.pow(2, attempt) * 1000),
      );
    } catch (error) {
      if (attempt === maxRetries) {
        return { success: false, "error-codes": ["internal-error"] };
      }
    }
  }
```

```
}
```

Enhanced validation with custom checks

```
async function validateTurnstileEnhanced(
  token,
  remoteip,
  expectedAction = null,
  expectedHostname = null,
) {
  const validation = await validateTurnstile(token, remoteip);

  if (!validation.success) {
    return {
      valid: false,
      reason: "turnstile_failed",
      errors: validation["error-codes"],
    };
  }

  // Check if action matches expected value (if specified)
  if (expectedAction && validation.action !== expectedAction) {
    return {
      valid: false,
      reason: "action_mismatch",
      expected: expectedAction,
      received: validation.action,
    };
  }

  // Check if hostname matches expected value (if specified)
  if (expectedHostname && validation.hostname !== expectedHostname) {
    return {
      valid: false,
      reason: "hostname_mismatch",
      expected: expectedHostname,
      received: validation.hostname,
    };
  }

  // Check token age (warn if older than 4 minutes)
  const challengeTime = new Date(validation.challenge_ts);
  const now = new Date();
  const ageMinutes = (now - challengeTime) / (1000 * 60);

  if (ageMinutes > 4) {
```

```
    console.warn(`Token is ${ageMinutes.toFixed(1)} minutes old`);
  }

  return {
    valid: true,
    data: validation,
    tokenAge: ageMinutes,
  };
}

// Usage
const result = await validateTurnstileEnhanced(
  token,
  remoteip,
  "login", // expected action
  "example.com", // expected hostname
);

if (result.valid) {
  // Process the request
  console.log("Validation successful:", result.data);
} else {
  // Handle validation failure
  console.log("Validation failed:", result.reason);
}
```

API response format

Successful response | Failed response

Example

```
{
  "success": true,
  "challenge_ts": "2022-02-28T15:14:30.096Z",
  "hostname": "example.com",
  "error-codes": [],
  "action": "login",
  "cdata": "sessionid-123456789",
  "metadata": {
    "ephemeral_id": "x:9f78e0ed210960d7693b167e"
  }
}
```

Response fields

Field	Description
`success`	Boolean indicating if validation was successful
`challenge_ts`	ISO 8601 timestamp when the challenge was solved
`hostname`	Hostname where the challenge was served
`error-codes`	Array of error codes (if validation failed)
`action`	Custom action identifier from client-side
`cdata`	Custom data payload from client-side
`metadata.ephemeral_id`	Device fingerprint ID (Enterprise only)

Error codes reference

Error code	Description	Action required
`missing-input-secret`	Secret parameter not provided	Ensure secret key is included

Error code	Description	Action required
`invalid-input-secret`	Secret key is invalid or expired	Check your secret key in the Cloudflare dashboard
`missing-input-response`	Response parameter was not provided	Ensure token is included
`invalid-input-response`	Token is invalid, malformed, or expired	User should retry the challenge
`bad-request`	Request is malformed	Check request format and parameters
`timeout-or-duplicate`	Token has already been validated	Each token can only be used once
`internal-error`	Internal error occurred	Retry the request

Implementation

Example implementation

```
class TurnstileValidator {
  constructor(secretKey, timeout = 10000) {
    this.secretKey = secretKey;
    this.timeout = timeout;
  }

  async validate(token, remoteip, options = {}) {
    // Input validation
    if (!token || typeof token !== "string") {
      return { success: false, error: "Invalid token format" };
    }

    if (token.length > 2048) {
      return { success: false, error: "Token too long" };
    }

    // Prepare request
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), this.timeout);

    try {
      const formData = new FormData();
      formData.append("secret", this.secretKey);
      formData.append("response", token);

      if (remoteip) {
        formData.append("remoteip", remoteip);
      }

      if (options.idempotencyKey) {
        formData.append("idempotency_key", options.idempotencyKey);
      }

      const response = await fetch(
        "https://challenges.cloudflare.com/turnstile/v0/siteverify",
        {
          method: "POST",
          body: formData,
          signal: controller.signal,
        },
      );
```

```
      const result = await response.json();

      // Additional validation
      if (result.success) {
        if (
          options.expectedAction &&
          result.action !== options.expectedAction
        ) {
          return {
            success: false,
            error: "Action mismatch",
            expected: options.expectedAction,
            received: result.action,
          };
        }

        if (
          options.expectedHostname &&
          result.hostname !== options.expectedHostname
        ) {
          return {
            success: false,
            error: "Hostname mismatch",
            expected: options.expectedHostname,
            received: result.hostname,
          };
        }
      }

      return result;
    } catch (error) {
      if (error.name === "AbortError") {
        return { success: false, error: "Validation timeout" };
      }

      console.error("Turnstile validation error:", error);
      return { success: false, error: "Internal error" };
    } finally {
      clearTimeout(timeoutId);
    }
  }
}
```

```
// Usage
const validator = new TurnstileValidator(process.env.TURNSTILE_SECRET_KEY);

const result = await validator.validate(token, remoteip, {
  expectedAction: "login",
  expectedHostname: "example.com",
});

if (result.success) {
  // Process the request
} else {
  // Handle failure
  console.log("Validation failed:", result.error);
}
```

Testing

You can test the dummy token generated with testing sitekey via Siteverify API with the testing secret key. Your production secret keys will reject dummy tokens.

Refer to Testing for more information.

Best practices

Security

- Store your secret keys securely. Use environment variables or secure key management.
- Validate the token on every request. Never trust client-side validation alone.
- Check additional fields. Validate the action and hostname when specified.
- Monitor for abuse and log failed validations and unusual patterns.
- Use HTTPS. Always validate over secure connections.
- Only call the Siteverify API in your backend environment. If you expose the secret key in the front-end client code to call Siteverify, attackers can bypass the security check. Ensure that your client-side code sends the validation token to your backend, and that your backend is the sole caller of the Siteverify API.

Performance

- Set reasonable timeouts. Do not wait indefinitely for Siteverify responses.
- Implement retry logic and handle temporary network issues.
- Cache validation results for the same token, if it is needed for your flow.
- Monitor your API latency. Track the Siteverify response time.

Error handling

- Have fallback behavior for API failures.
- Use user-friendly messaging. Do not expose internal error details to users.
- Properly log errors for debugging without exposing secrets.
- Rate limit to protect against validation flooding.

WAS THIS HELPFUL?

Yes No

Previous
← Widget configurations

Next
Mobile implementation →

Last updated: May 5, 2026

© 2026 Cloudflare, Inc. • Privacy Policy • Terms of Use • Report Security Issues • Trademark • Cookie Preferences